Comparing KV Cache Eviction Policies

Hui-Kang Tong | GStar Bootcamp 2026 Individual Project Report
27 October 2026

Abstract

Most LLMs keep global attention layers, whose memory grows with every token in the context.

Cache eviction bounds that memory by dropping the keys and values of some past tokens, and "global" attention is now calculated with the remaining KV cache. I compare five cache eviction policies on gpt-oss-20b. I measure them two ways, by replaying twenty frozen full-cache AIME runs as reference traces and by running AIME problems under a fixed cache.

I also describe using cache eviction in ARC-AGI-3. I achieved greater decode throughput than the Milestone 2 prize-winning submission at parity in score per token. I inspected which tokens are evicted. As of report submission, I am in 25th place in ARC-AGI-3.

1 Introduction

The memory of a global attention layer grows with context length, because every past token keeps a key and a value. Sliding window attention and linear attention reduce how many layers pay that cost, but models still keep global attention layers, and inference memory usage still scales with context length.

Not every cached token matters. Some tokens hardly influence the output, because their attention weight is near zero. This is similar to human memory. Humans can hold a long conversation without remembering every word.

2 Candidate eviction policies

An eviction policy decides which cache entries to drop. Every policy below drops whole key and value entries, and all of them decide at the same cadence, once every 2048 tokens. Each keeps a protected prefix of k_1 tokens and a protected recent window of k_2 tokens, and fills the remaining k_3 by its own rule.

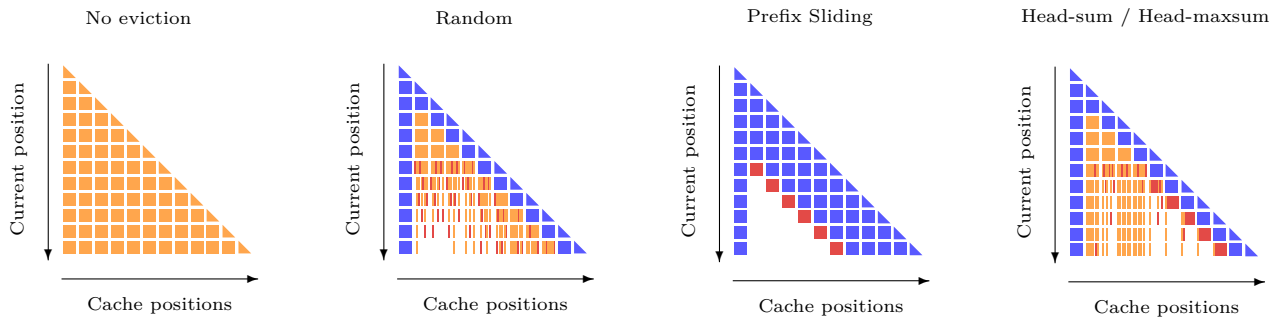


Figure 1. Illustrative causal retention maps, with current position on rows and cache positions on columns. Blue marks protection, orange other retention, red the entries evicted at the end of that chunk, and white entries already gone, with blank future positions and equal retained area across eviction policies.

Prefix Sliding [1]. Retain the cache of the first k_1 tokens and the last $k_2 + k_3$ tokens.

Random Attention [2]. Retain the cache of the first k_1 tokens, the last k_2 tokens and a random k_3 of the tokens in between. The random scores are drawn afresh at every boundary, so an old token faces the draw many times and its chance of surviving falls with its age.

Reservoir [3]. The same, except that each token draws one random score when it arrives and keeps it. Keeping the k_3 highest fixed scores among everything seen so far is reservoir sampling, so the retained middle is a uniform sample of the whole history and an early token is as likely to remain as a recent one.

Head-sum [4]. Retain the cache of the first k_1 tokens and the last k_2 tokens, and evict tokens based on attention head sum, retaining k_3 tokens. This is the attention-weight sum of H_2O, which accumulates normalized attention over time.

Head-maxsum [4]. Retain the cache of the first k_1 tokens and the last k_2 tokens, and evict tokens based on attention max sum, retaining k_3 tokens. This is my adaptation of the same H_2O score, taking the largest head weight at each query instead of their sum.

To keep the comparison fair, the total amount of cache allowed is constant between the policies. I set $k_1 = k_2 = k_3/2$.

2.1 Implementation on gpt-oss-20b

Each of the 12 full-attention layers and each of its 8 KV heads chooses its own survivors, as in Random Attention [2]. The counters of Head-sum and Head-maxsum are kept per KV head, as the sum or the maximum over the 8 query heads that share it. At a boundary every head's survivors are copied into one shared set of slots. The copies need no extra memory, because the slots they read from and write to are disjoint. The 12 sliding-window layers keep their own 128-token window and are never evicted.

2.2 Parity metrics on reference traces

There are two ways to measure the performance of an eviction policy. One is to replay frozen reference traces offline and compare each policy against full cache. The other is to run the task itself and measure accuracy.

I use the 20 full-cache AIME [5] runs of the online sweep in Section 2.3 as the reference traces, problems 4, 11, 13, 26 and 29 with four seeds each, including the two that reached the 130,000-token limit without an answer. Each trace is fed back to the model as a prompt in 2048-token chunks, and at every boundary the policy hides its victims from the full-attention layers. Each metric below is averaged over the 20 traces.

I calculate three parity metrics, each comparing a policy against full cache on the same trace.

Logprob difference. If eviction changed nothing, the model would give every chosen token the same logprob as it does with full cache. The metric is the mean absolute difference between the logprob of each chosen token with and without eviction.

Attention weight difference. This is the attention weight the unevicted model puts on keys the policy has already evicted, averaged over queries, heads and the full-attention layers.

First layer output difference. This is the relative L2 distance between the attention outputs of the first full-attention layer with and without eviction.

	Logprob difference		Attention weight difference		First layer output difference	
Candidate	8K	16K	8K	16K	8K	16K
Random	0.087 ±0.017	0.048 ±0.014	0.132 ±0.020	0.065 ±0.018	**0.069** ±0.009	0.032 ±0.008
Reservoir	0.095 ±0.020	0.051 ±0.016	0.155 ±0.025	0.083 ±0.024	0.081 ±0.012	0.042 ±0.012
Prefix Sliding	0.097 ±0.019	0.054 ±0.015	**0.120** ±0.019	**0.045** ±0.013	0.076 ±0.009	**0.022** ±0.007
Head-sum	**0.080** ±0.017	0.043 ±0.013	0.128 ±0.026	0.062 ±0.021	0.072 ±0.011	0.038 ±0.010
Head-maxsum	**0.080** ±0.017	**0.042** ±0.013	0.128 ±0.026	0.062 ±0.021	0.071 ±0.011	0.038 ±0.010

Table 1. Parity metrics for each eviction policy at retained caches of 8K and 16K on the 20 full-cache AIME traces. Bold marks the lowest value in each column. Each ± gives the half-width of the 95% confidence interval over the traces.

Table 1 gives the three metrics at an 8K and a 16K retained cache, and they do not agree on which policy is best.

Head-sum and Head-maxsum give the smallest logprob difference, yet neither has the lowest attention weight or first layer output difference.

2.3 Metrics calculated online

I also compare the performance of gpt-oss-20b under a limited retained cache.

The sweep compares every policy at the same retained KV-cache budgets. No tools are available to the model.

The sweep uses the long problems 4, 11, 13, 26, 29, where full cache needed 55K to 67K tokens. Each runs at 2, 4, 8, 16, 32K retained tokens with four seeds, and full cache, rerun with the same seeds, answers 18 of 20. Appendix A records the model and serving details.

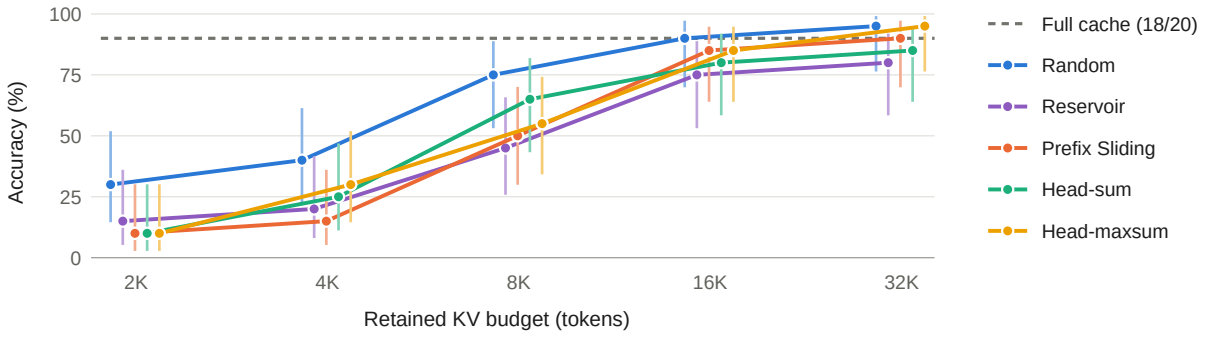


Figure 2. Accuracy of each eviction policy against retained KV budget, with the full-cache reference as a flat line at 90.0%. Bars are 95% Wilson intervals over 20 runs per point, five problems with four seeds each.

With about a quarter of the cache, the policies come close to full-cache accuracy. The choice of eviction policy matters little. At 2K to 8K, Random Attention scores highest.

3 Use in ARC-AGI-3 competition

Cache eviction is one of the main strategies in my ARC-AGI-3 [6] submission.

In the official ARC-AGI-3 competition, participants submit a piece of code that is run on a Google Cloud g4-standard-48 instance with one NVIDIA RTX PRO 6000. The popular model in ARC-AGI-3 is the 125B-parameter Qwen3.8-Flash-Next [7], run from the W4A16 quantization [8]. It also carries 51B parameters of n-gram per-layer embeddings (PLE), about 100 GB in BF16, which are offloaded to host memory.

3.1 Implementation on Qwen3.8-Flash-Next

Qwen3.8-Flash-Next uses Qwen Sparse Attention. Every four tokens are indexed together as a block, which a query either reads whole or skips, and each query reads only the 512 blocks it selects. Eviction therefore happens at the block level instead of the token level. I keep a counter per block, scored by the maximum post-softmax attention over its four tokens and all 24 query heads, counted after a delay of 128 tokens, and at most 512 blocks are evicted at a boundary. Each sparse attention layer chooses its own victims.

3.2 Analysis of eviction patterns

I built an eviction viewer at https://tonghuikang.github.io/cache-eviction/arc-agi-3 that shows one capture from AR25, an ARC-AGI-3 game, token by token. The capture covers 79,720 prompt tokens over 12 captured layers, made up of 64,885 text tokens and 38 images. Each token is shaded by how many of the 12 layers still hold its block.

Repetitive text is evicted first. The longest spans that no layer keeps are the per-pixel diffs the game sends back, hundreds of tokens of coordinate lists and board rows drawn as dots and question marks, such as `50 33 9 -> 10`, `50 34 9 -> 10` and `50 35 9 -> 10` on successive lines (Figure B2).

Important events are retained. Of the 61,534 text tokens between the protected prefix and the recent window, 6,065 (9.9%) are still held by all 12 layers, and 74% of those are the model's own reasoning, such as `RIGHT` → `P -3 (left), B +3 (right), wall unchanged` (Figure B3).

Key image features are retained. A further 17% of those fully held tokens is what the harness sends back, including the statistics it prints for each frame, the bounding boxes and the per-region counts, such as `12-26 count 72 moved 3 pixels left` in the middle of the trace (Figure B3). The per-pixel diffs those statistics summarize are evicted, and a fresh one arrives every turn. In the board images themselves, the patches that cover the game pieces and the target stay in most layers, while most of the uniform background is evicted (Figure B4).

3.3 Performance

Cache eviction has four benefits for the submission.

Decouple the KV cache allowance from context length. A 128K-token sequence can be decoded on a 64K-token KV budget. This allows longer sequences to be decoded without compaction.

Run more sequences in parallel. A bounded cache per sequence means more of them fit in the same pool, which allows for greater decode throughput.

Avoid bin-packing. The Milestone 2 prize-winning solution had to plan in advance how many tokens each session would take, so that the sessions it ran together fitted the pool. A bounded cache per sequence removes that planning.

Skip the prefill when a level clears. The usual approach discards the trace at the end of a level and prefills the next one from scratch. With eviction the cache of the previous level is dropped and decoding continues.

Figure 3 compares Daniel Franzen's Milestone 2 prize-winning harness with my eviction-enabled run, following the comparison in [9]. Decode throughput stays above the Milestone 2 winning solution for almost the whole run, and the dips that mark prefill-heavy sessions are absent. Score per unit of wall-clock time is higher than the Milestone 2 solution, while score per token matches it.

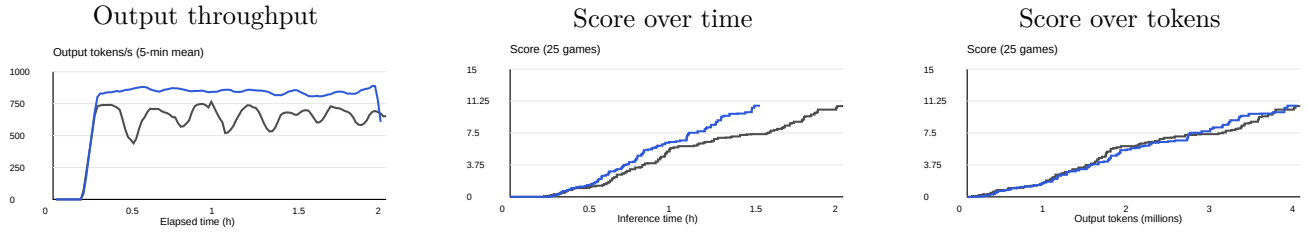


Figure 3. Recorded throughput and score comparisons on public games. Grey is the Milestone 2 winning harness and blue is my eviction run.

I was the first to reach 20 points. At the end of Milestone 2, I was in 7th place with a score of 20.53 on the official leaderboard. After integrating ideas from the Milestone 2 winning solution, I am scoring 37.92, currently in 25th place.

Conclusion and future work

I have compared cache eviction policies on gpt-oss-20b over a selected set of AIME problems, and described how I use cache eviction as a competitive strategy in ARC-AGI-3. On AIME, long reasoning needs 16K to 32K retained tokens before the policies come close to full cache. On ARC-AGI-3, eviction gives higher decode throughput at parity in score per token.

I hope cache eviction makes it into mainstream inference frameworks, because most of what it needs already exists inside the serving stacks but is not exposed. Inference frameworks should provide two options.

Cache budget and eviction policy. The caller should be able to set a cache budget smaller than the context length, and choose the policy that enforces it. Today the only way to bound the memory of a sequence is to bound its context, which also bounds what the model can see.

Cache eviction range. The caller should be able to name the entries to evict, overriding whatever the policy would have chosen. When a task finishes a stage, a defined cache range can then be dropped while decoding continues, without restarting and prefilling the rest from scratch.

References

[1] N. Muennighoff et al. Prefix Sliding for Efficient Test-Time Scaling. 2026. arXiv:2608.26070.

[2] H. Wang et al. Random Attention: Rethinking KV Cache Eviction for Efficient Reasoning. 2026. arXiv:2609.03430.

[3] J. S. Vitter. Random Sampling with a Reservoir. ACM Transactions on Mathematical Software 11(1), 1985.

[4] Z. Zhang et al. H_2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. 2023. arXiv:2306.14048.

[5] MathArena. AIME 2026. 2026. Hugging Face dataset.

[6] ARC Prize. ARC Prize 2026 – ARC-AGI-3 Competition. 2026. Competition overview.

[7] Qwen Team. Qwen3.8-Flash-Next. 2026. Official model repository.

[8] Intel. Qwen3.8-Flash-Next-W4A16-AutoRound. 2026. Hugging Face checkpoint.

[9] H.-K. Tong. Comparing the Milestone 2 winning solutions. September 2026. Kaggle discussion.

[10] OpenAI. Introducing gpt-oss. 2025. Model announcement.

A Model and measurement details

gpt-oss-20b is `openai/gpt-oss-20b` [10], revision `6cee5e8`, with MXFP4 expert weights and a bf16 KV cache. It runs on SGLang 0.5.21 with the Triton attention backend, PyTorch 2.13 and CUDA 13.0, patched by an overlay that edits five SGLang files and adds one runtime module.

The sweep, the full-cache runs and the parity metrics ran on one NVIDIA RTX PRO 6000 on Modal. The context limit is 131,072 tokens, generation is capped at 130,000 tokens, reasoning effort is high, and temperature is 1.

Up to 320 requests share the server, admitted while their worst-case cache reservations fit within 95% of its 2.89M-token pool, and no run was retracted. Each run decodes as a streaming session in calls that end on 2048-token boundaries, with sampling noise keyed on the seed and the position. Every prompt is shorter than 2048 tokens, and all evictions happen during decode.

Ranking and compaction take a median of 2.5 ms and 0.6 ms per boundary, over 15,979 evicting boundaries under full load.

The code will be made available in https://github.com/tonghuikang/cache-eviction.

ARC-AGI-3 comparison

Figure 3 compares my eviction run with the Milestone 2 winning solution on the 25 original ARC-AGI-3 [6] games. The two runs use slightly different harnesses. The code for my ARC-AGI-3 submission will be made available after the competition ends on 2 November 2026.

B The eviction viewer

The viewer shades each token by how many of the 12 layers still hold its block. Figure B1 shows a span of reasoning around token 11,300 in retained focus, from https://tonghuikang.github.io/cache-eviction/arc-agi-3/?focus=retained&pos=11300.

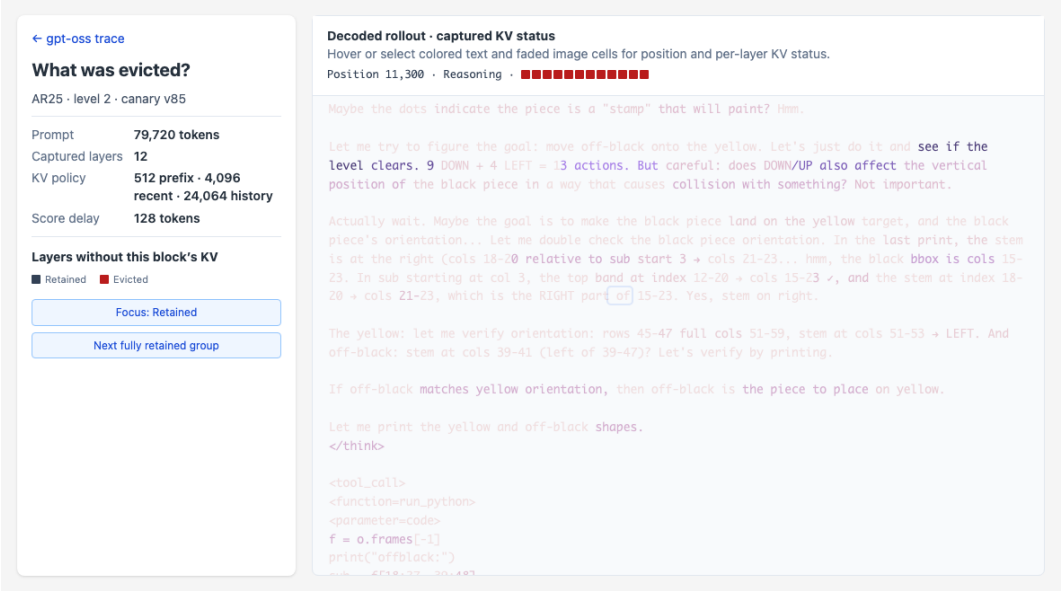


Figure B1. Retained focus. What the layers kept is dark and what they dropped fades into the page.

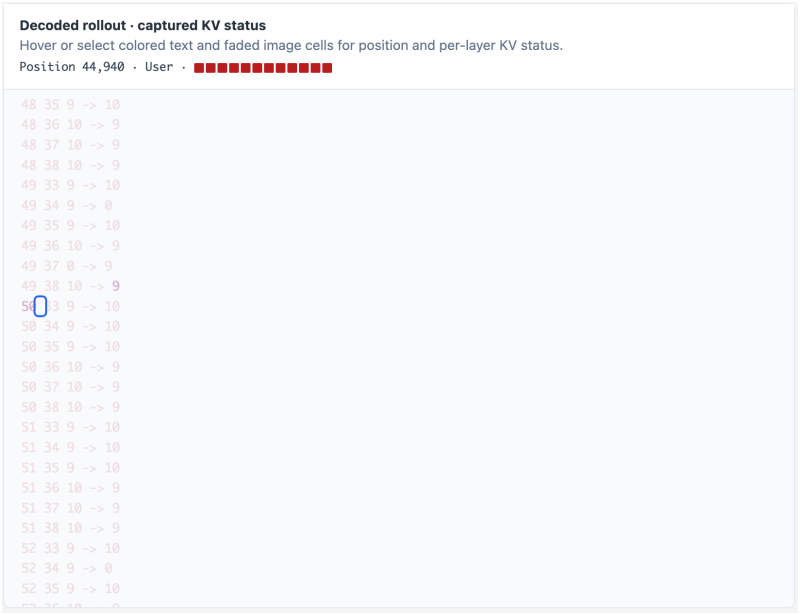


Figure B2. The per-pixel diff quoted in Section 3.2, around token 44,940. No layer holds these lines. Open in the viewer.

Decoded rollout · captured KV status

Hover or select colored text and faded image cells for position and per-layer KV status.

Position 47,078 · Reasoning ·

off-black (4)
row 18-29 col 12-26 count 72 moved 3 pixels left
black (5)
row 18-29 col 48-62 count 64 moved 3 pixels right containing 8x white (0) count 1 point

57 actions, 1 in last
58 frames, 1 in last
</tool_response><|im_end|>
<|im_start|>assistant
<think>
Confirmed: RIGHT → P -3 (left), B +3 (right), wall unchanged. So pressing RIGHT moves P LEFT and B RIGHT (they move apart). LEFT → P -6, wall -3, B unchanged?? That contradicts a mirrored pair.

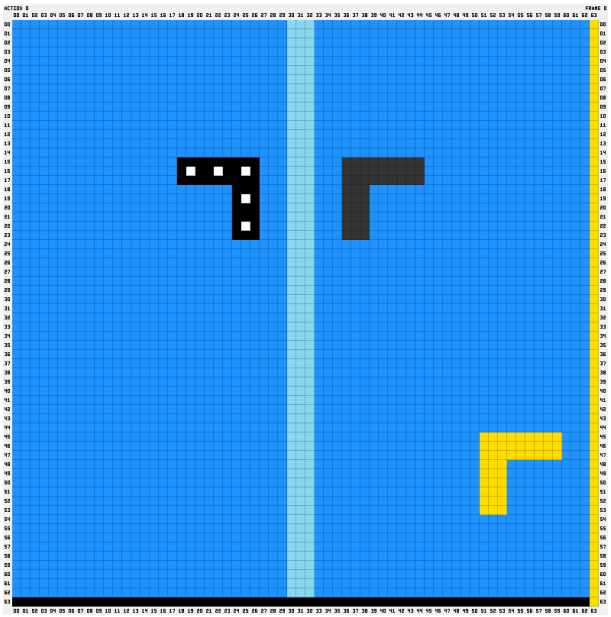
Hmm wait – maybe on LEFT, P moves -3 AND the wall moves -3, and P is drawn... no, P's diff: new P cells at cols 12..20 (rows 18,27-29) and old P removed at 18..23 – consistent with a -6 shift? Let's verify: P at origin 15, shape spans 15..29. After: origin 9, spans 9..23. Yes -6.

Hmm, unless the LEFT press did something else: maybe pressing LEFT moved P left by 3 AND the wall left by 3, and there was ALSO a preceding queued state? No – diff between frames 0 and 3.

But hold on: maybe LEFT and RIGHT both move P left?! In level 1: RIGHT: P -3; LEFT: P +3. So in level 1 LEFT moved P right. So no.

Actually, maybe the amount differs per level, and in level 2 the mechanic is different: maybe there are two independent "crawlers" driven by the same key press: hmm.

Figure B3. The model's conclusion quoted in Section 3.2, around token 47,078, in the middle of the trace. At least 11 of the 12 layers hold the conclusion and the two sentences after it, while most of the reasoning around them fades. Open in the viewer.



Original recorded image



Retained-token emphasis

Figure B4. The first board image of the capture, tokens 1,631 to 2,255, where each 32-by-32-pixel patch is one image token. The right panel fades each patch by how many of the 12 layers still hold its block. Open in the viewer.